There was a plutonium probe?!

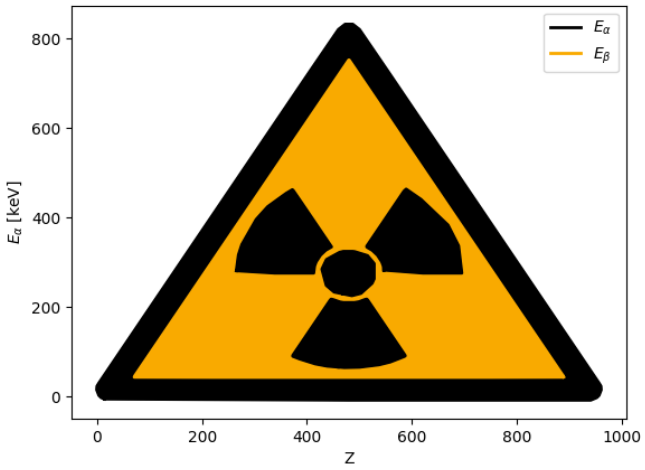


Figure 0.1: Attention, Attention! Dear comrades! The lab council reports that [...] an unfavorable radiation situation is developing in the lab rooms. [...][6] [7]

Analysis of experiment 256 - X-ray fluorescence

"Physikalisches Praktikum 2.2"

in the physics departement of

university Heidelberg

by

Benjamin Kleijn

August 11, 2026

lab date	28.04.2026
group	9
tutor	Frisco Grace

Contents

List of Figures

List of Tables

1. Introduction

1.1. Motivation

X-ray radiation is one important type of radiation used in medical applications due to its ability to penetrate solid substances, e.g. identifying broken bones. Additionally, it can be used in material science to analyze chemical elements and material testing (evaluate its structural stability). Like other radiation types, it poses as an interesting subject for physics, its mechanics need to be understood in order to provide the necessary precision for medical purposes. The following experiment introduces X-ray radiation and hints at interesting concepts such as detecting methods with semiconductors. But most importantly it provides a useful insight on atomic behaviour after excitation by X-ray radiation, a nice experimental addition to atomic physics studied in the corresponding quantum mechanics lectures.

1.2. Experiment goals

This experiment is done to study X-ray mechanics: production/emission, detection and interaction with materials. Different types of metal plates under X-ray radiation are studied, explicitally the experiment aims to:

- analyze the K_{α^-} and K_{β^-} emission lines of said metals
- compare the values with the theoretical model of "Mosley's law"
- determine the composition of different alloys by studying their emission lines

1.3. Setup and devices

The X-ray chamber consists of a X-ray tube (a heated electrode to create fast electrons which are accelerated towards the anode by an electric field, after hitting it, they are slowed down fastly by the anodes material emitting X-ray radiation as Bremsstrahlung). The created X-ray spectrum is focused onto the probe, a small metal plate. The resulting secundary X-ray radiation caused by atomic transitions (which themselves are driven by the primary X-ray radiation) in the probe is analyzed by an energy detector coupled with an amplifier to generate voltages corresponding to certain transitions, which are then processed by a multiple channel analyzer and displayed by the measurement programm.

1. X-ray chamber with X-ray tube
2. X-ray energy detector (PIN diode)
3. multiple channel analyzer
4. metal probes in plate form

5. desktop PC with CassyLab (measurement programm)

Figure 1.1: experiment setup[1]

1.4. Theoretical background[1]

1.4.1. X-ray fluorescence

X-ray radiation is high energy electromagnetic radiation, which can eject an atom's inner (strongly bound) electrons, analogous to the photoeffect. The resulting holes are filled by electrons of higher shells/states with larger quantum number, emitting a photon of the corresponding energy difference in the process. An electron of the second shell (K_α radiation) is most likely to fill the ejected electron's place, less likely is an electron of the third shell (K_β radiation). The terms originate by using the final shell letter, K, and the difference between initial and final shell, $\alpha \mathrel{\hat{=}} 1$, $\beta \mathrel{\hat{=}} 2$.

1.4.2. Moseley's law

Starting at the Rydberg-formula (solution to the Schrödingers equation for Hydrogen), one needs to take account for effects to describe the transition energy ΔE for heavier objects correctly. Looking at the transition between the states with quantum number $n = 2 \to n = 1$ and their corresponding energys $E_1; E_2$, one introduces a shielding constant σ_{12} to describe the

shielding of the cores charge by other electrons. The effective core nuclear charge number is corrected by $Z_{\text{eff}} = Z - \sigma_{12}$. Therefore one arrives at:

$$E := \Delta E = E_2 - E_1 = chR_\infty (Z - \sigma_{12})^2 (\frac{1}{n_1^2} - \frac{1}{n_2^2}) \tag{1.1}$$

(where c is the speed of light, h planck's constant)
Leading to Moseley's law by taking the square root and substituting $chR_\infty = E_{Ry} = 13.6\,\text{eV}$:

$$\sqrt{\frac{E}{E_R}} = (Z - \sigma_{12}) \sqrt{\frac{1}{n_1^2} - \frac{1}{n_2^2}} \tag{1.2}$$

We neglect finestructure effects.
If we examine the K_α line, $\sigma_{12} = 1$ and Moseley's law breaks down to this simple form:

$$\sqrt{\frac{E}{E_R}} = (Z - 1) \sqrt{\frac{1}{3} - \frac{1}{4}} \tag{1.3}$$

1.4.3. Energy detector

The device responsible for measuring the emitted X-ray frequencies is the energy detector: a PIN-diode. One can explain the system in these simplified terms: one layer[1] of an n-type semiconductor ("negative" side containing freely moving electrons) connected with one layer of a p-type semiconductor ("positive" side containing freely moving electron holes) is creating a pn-transition: At its electrical junction, electrons and electron holes diffuse to the respective other side thus creating a depletion zone without moving charge. This allows electric current to pass through the junction only in one direction. This zone can be enlargened by applying external voltage. This region is of great interest, because whenever a X-ray photon hits the depletion zone, it can get absorbed whilst releasing a photoelectron. This photon is scattered on the crystal structure of the junction, creating new electron-hole pairs in the process. These are drawn away by the external voltage and the signal, proportional to the incoming X-ray photons, is measured.

The multi-channel-system categorizes the signals in different channels. Each channel holds information about the count of appearance of its signal (event counts). Wavelengths, that are produced more frequently in the metal's atomic transitions are therefore associated with a higher count.

The last necessity to interpret the measured data is to calibrate/gauge the energy scale: This is done by measuring the X-ray fluorescence(signals) of a well known material (iron and molybdän in this case), and attributing its results to theoretical/characteristic values.

[1] Sir Layer Cake, 1875-1941

2. Procedure and data protocol

2.1. Measurement methods

We can measure the emission spectrum pretty easily (the whole apparatus is already set up): Just place the metal plates onto their designated spot in the X-ray beam, close the chamber door and start the X-ray radiation by pressing one button. The results are displayed on the computer after setting up the CassyLab programm.

The metal probes, see fig. 2.1, are examined by recording the counts for 3 minutes. After calibrating with iron and Molybdenum, the channels and their counts are displayed with the correct energy, now gaussian curves can be fitted to he K_α, K_β peaks.

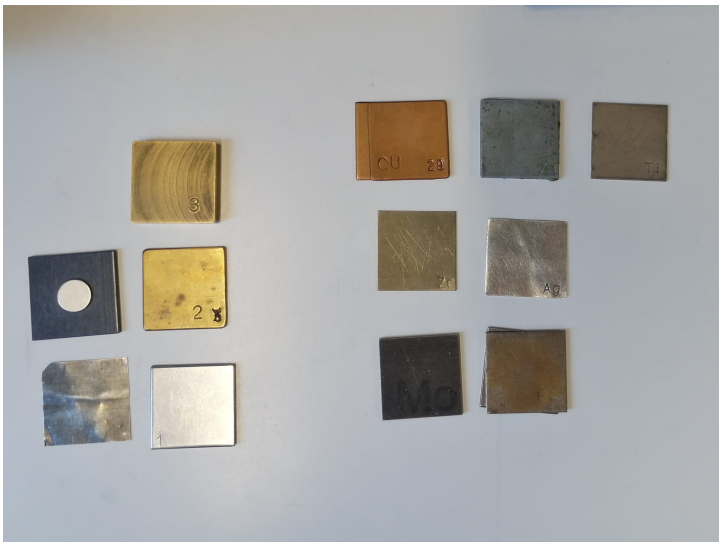


Figure 2.1: right side: used metal probes; left side: unknown alloys

The programm allows to insert the theoretical/literature energy values/emission lines of different elements into the graph. This feature is used to roughly fit an elements characteristic lines to the peaks in the spectrum of unknown alloys, resulting in an approximation of the alloys' composition. An example is shown in fig. 3.7.

2.2. Lab protocol / data sheet

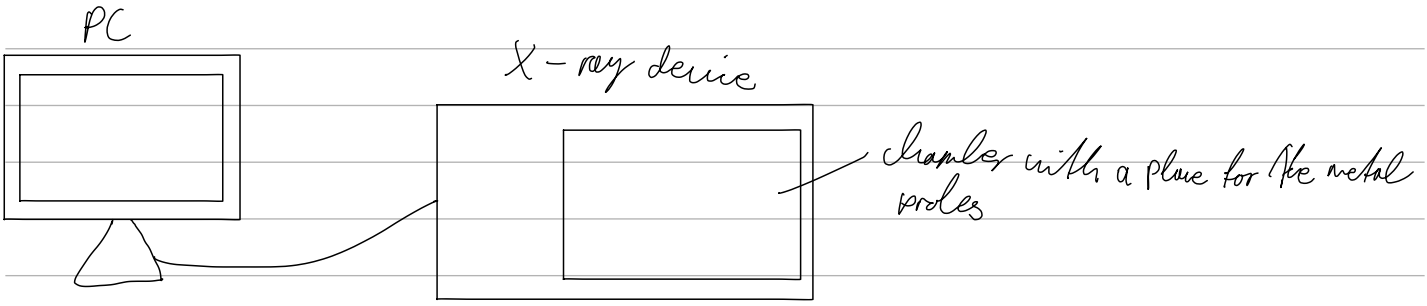
Experiment 256: X-ray fluorescence

Paul Zornmüller, Tobit Kleijn

Equipment

- X-ray device with an X-ray detector
- Multichannel analyzer
- several samples of metal (Zr, Zn, Ni, Ti, Ag, Fe, Cu, Mo, Zr brass, stainless steel, magnet, unknown alloy)
- computer

Setup



Task 1: calibration of the X-ray device

We set the measurement parameters as the following:

number of channels: 512
negative pulses
amplification factor: -2.5
measurement time: 180s
voltage: 35kV
current: 1mA

We also need to calibrate the x-axis. To do this, we fit a Gaussian curve to the $K\alpha$-lines of Fe and Mo, of which we know the energies ($E_{\alpha,Fe} = 6.4$ keV, $E_{\alpha,Mo} = 17.48$ keV). We record their channels as $\mu_{\alpha,Fe} = 30$, $\sigma_{\alpha,Fe} = 2$, $\mu_{\alpha,Mo} = 216$ and $\sigma_{\alpha,Mo} = 2$, and can thus calibrate the x-axis.

Task 2: Measuring line energies as Gaussian curves

We make the following measurements:

Table 1: Energies of the Gaussian fits for every line

Line	μ [keV]	σ [keV]	
Zr, α	15,84	0,18	
Zr, β	17,69	0,16	
Zn, α	8,72	0,19	
Zn, β	9,66	0,19	
Fe, α	6,44	0,18	
Fe, β	7,04	0,32	
Mo, α	17,49	0,19	
Mo, β	19,56	0,16	
Ni, α	7,52	0,19	
Ni, β	8,28	0,27	
Cu, α	8,10	0,19	
Cu, β	8,97	0,25	
Ti, α	4,61	0,17	
Ti, β	–	–	There was barely a bump in the curve, and we could not fit a satisfactory Gaussian
Ag, α	21,84	0,19	
Ag, β	24,49	0,23	

Measurement order: Cu, Ag, Zn, Ti, Zr, Ni, Mo, Fe

Task 3: Compositional analysis of alloys

We measure the spectrums of our alloy samples, although we do not analyze them numerically. After doing so, we use the programs built-in values for an elements X-ray lines to analyze their composition.

stainless steel: Fe, Ti, V, Mn
brass: Cu, Zn
magnet: Y, Cu, Zn, Ir, Mn
unknown alloy: Cu, Ni, (Hg (?)

3. Data analysis

Equations for statistical analysis[1]

Varianzen: Für die statistische Auswertung von n Messwerten $\{x_i\}_{i=1}^n$ werden folgende Größen definiert:

Arithmetisches Mittel	$\bar{x} = \text{Mean}(\{x_i\}_{i=1}^n) = \frac{1}{n}\sum_{i=1}^{n} x_i$	(S.1)
Varianz	$\sigma^2 = \frac{1}{n-1}\sum_{i=1}^{n}(x_i - \bar{x})^2$	(S.2)
Standardabweichung	$\sigma = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(x_i - \bar{x})^2}$	(S.3)
Fehler des Mittelwerts	$\Delta\bar{x} = \frac{\sigma}{\sqrt{n-1}}$	(S.4)

Fehlerfortpflanzung: Der Fehler einer Funktion $f(x_1, \ldots, x_N)$, die von den Variablen $\{x_i\}_{i=1}^N$ abhängt, wird wie folgt bestimmt:

Gauß'sche Fehlerfortpfl.	$\Delta f = \sqrt{\sum_{i=1}^{N}\left(\frac{\partial f}{\partial x_i}\Delta x_i\right)^2}$	(S.5)
relativer Fehler von $f = \prod_{i=1}^N x_i^{\alpha_i}$	$\frac{\Delta f}{\lvert f\rvert} = \sqrt{\sum_{i=1}^{N}\left(\frac{\alpha_i \Delta x_i}{x_i}\right)^2}$	(S.6)

Ergebnissignifikanz: Resultate eines Experiments bzw. Berechnung a_{exp} und die entsprechenden Literaturwerte a_{lit} werden folgend verglichen:

Absolute Abweichung	$A = \lvert a_{\text{lit}} - a_{\text{exp}}\rvert$	
	$\Delta A = \sqrt{(\Delta a_{\text{lit}})^2 + (\Delta a_{\text{exp}})^2}$	(S.7)
Relative Abweichung	$R[\%] = \frac{A}{a_{\text{lit}}} \cdot 100$	(S.8)
Signifikanz	$S[\sigma] = \frac{A}{\Delta A} = \frac{\lvert a_{\text{lit}} - a_{\text{exp}}\rvert}{\sqrt{\Delta a_{\text{lit}}^2 + \Delta a_{\text{exp}}^2}}$	(S.9)

Fitgüte: Für einen Fit mit Funktionswerten $\{F_{a_1,\dots,a_m}(x_i)\}_{i=1}^n$ und Fitparametern $\{a_j\}_{j=1}^m$ an die experimentellen Messwerte $\{y_i\}_{i=1}^n$ mit Fehler $\{\Delta y_i\}_{i=1}^n$ kann die Güte des Fits mit diesen Größen analysiert werden:

χ^2 Summe	$\chi^2 = \sum_{i=1}^{n} \left(\frac{F(x_i) - y_i}{\Delta y_i} \right)^2$	(S.10)
Freiheitsgrade	$f = n - m$	(S.11)
normierte reduzierte χ^2 Summe	$\chi^2_{\text{red}} = \frac{\chi^2}{f}$	(S.12)
Fitwahrscheinlichkeit	$p = 1 - \texttt{chi2.cdf}(\chi^2, f)$	(S.13)

Hierbei wird das Modul `chi2` von `scipy.stats` benutzt.

3.1. K_α lines

Mean μ and variance σ of gaussians curves fitted on the primary and secondary spectral peaks in fig. 3.1 are used for determining the energies E_α, E_β and their respective errors.

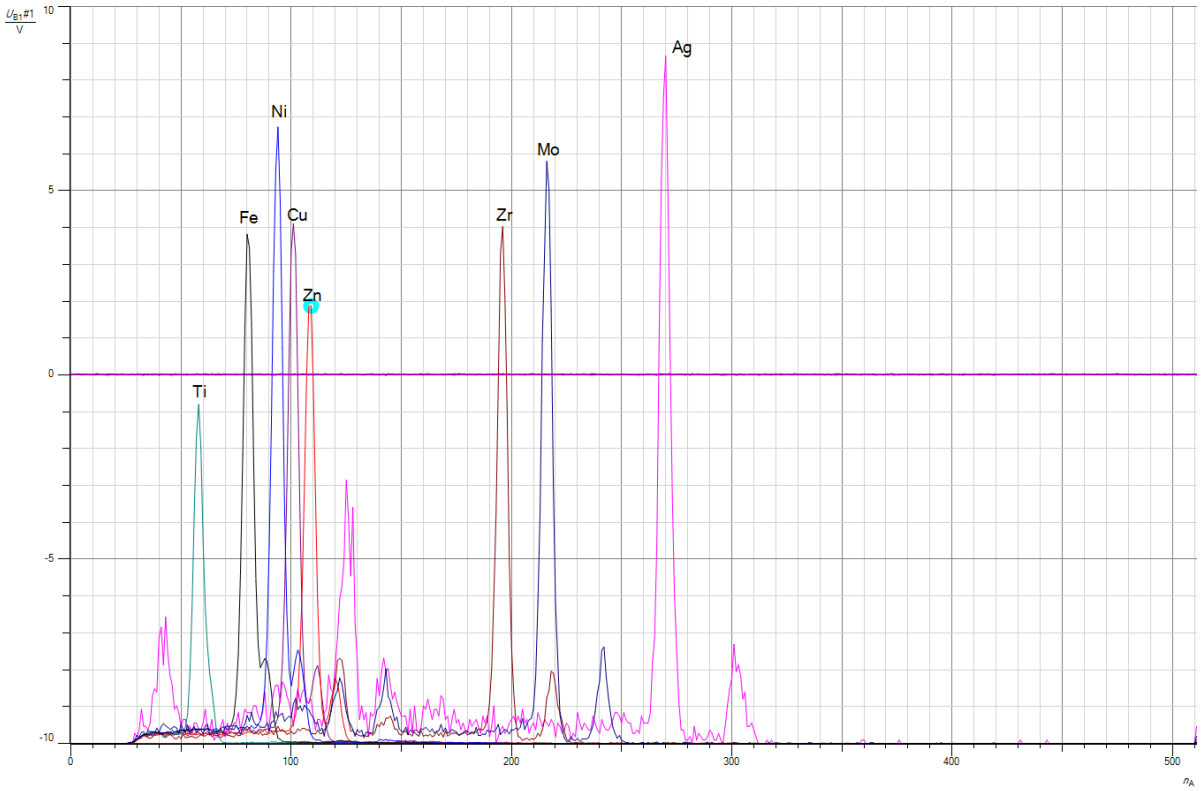


Figure 3.1: spectrums of all known metals

As eq. (1.2) suggests, there is a linear correlation of E to Z:

$$\sqrt{E} = \sqrt{E_R}(Z - \sigma_{12})\sqrt{\frac{1}{n_1^2} - \frac{1}{n_2^2}} \tag{3.1}$$

Thus, by plotting the $\sqrt{E_\alpha}$ energies against the charge number (known because the element is known, charge number is shown in the Abschnitt A.(Periodic table)), one can determine an approximation for E_R and σ_{12} by using them as fit parameters. The errors for $\sqrt{E_\alpha}$ are calculated as followed: $\Delta(\sqrt{E_\alpha}) = \frac{\sigma_\alpha}{2\sqrt{E_\alpha}}$. One arrives at this graph:

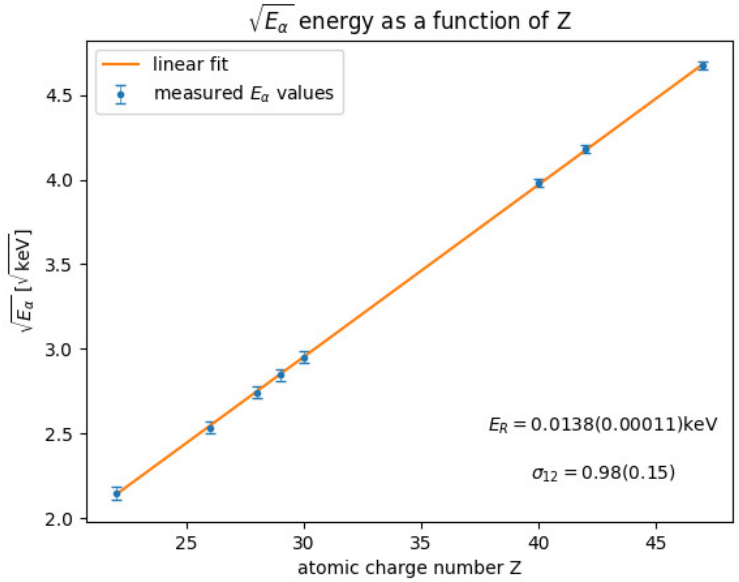


Figure 3.2: linear fit of $\sqrt{E_\alpha}$ against Z

By extracting the fit parameters and calculating the rooted energy back into its desired form:

$$E_R = \sqrt{E_R}^2 \qquad \Delta E_R = 2\sqrt{E_R}\Delta \cdot (\sqrt{E_R}) \tag{3.2}$$

one finds the values already shown in the graph.

$$E_R = (13.80 \pm 0.11)\,\text{eV} \qquad \sigma_{12} = (0.98 \pm 0.15)$$

3.2. K_β lines

The exact same procedure (for $n_f = 1, n_i = 3$) is done for the measured K_β lines. Because it wasn't possible to find a K_β line for Titanium (Ti), it is not part of the data this time. (if only the Titanic could avoid icebergs like Titanium is avoiding measurements)

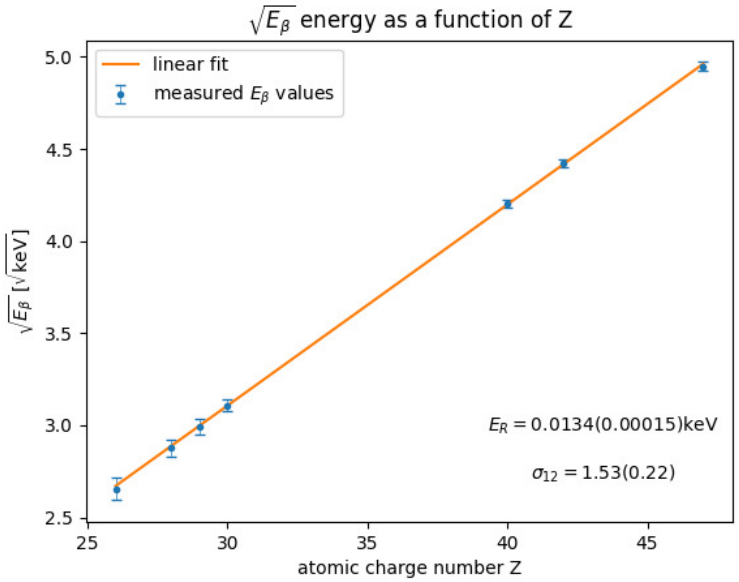


Figure 3.3: linear fit of $\sqrt{E_\beta}$ against Z

$$E_R = (13.30 \pm 0.15)\,\text{eV} \qquad \sigma_{13} = (1.53 \pm 0.22)$$

3.3. *Updated part:* Alloys

As one can see in the exemplary graph shown below (the rest are at the end of this section), the alloy's spectrums were taken seperately and corresponding energy lines of elements from the periodic table were fitted to determine their composition. One alloy was secured by plastic stripes/adhesive tape, probably it was present in a dusty form, it was certainly not in a solid metal crystal form in one big block like the others.

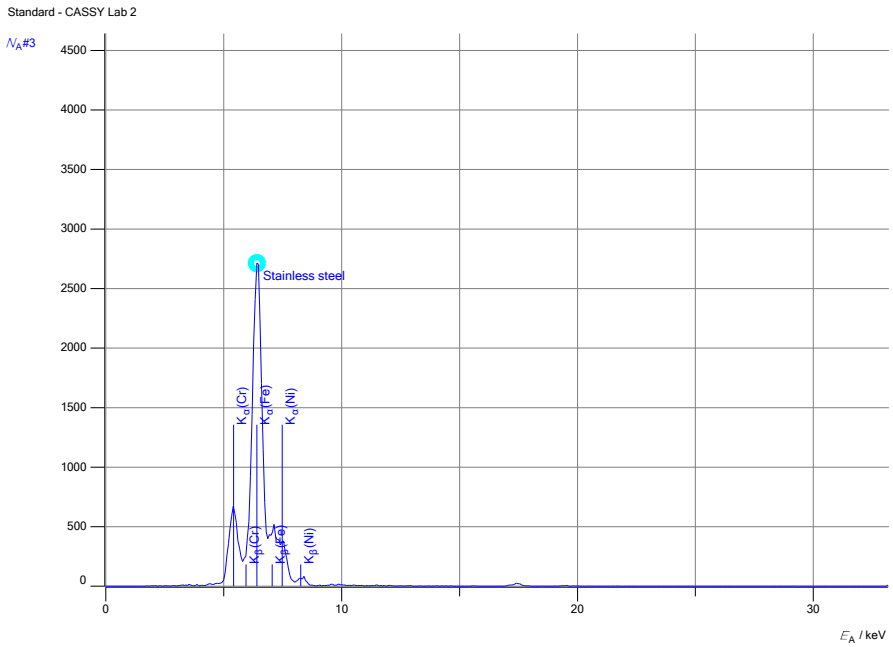


Figure 3.4: stainless-steel

The composition, that could be visually identified as common alloys, present itsel as:

- stainless steel: Fe, Ni, Cr
- brass: Cu, Zn
- magnet: Fe, W, Ti, Br
- unknown alloy: Cu, Pb, Ge, As

Brass was identified correctly, it consists generally of 2/3 copper (Cu) and 1/3 of zinc (Zn)[2]. Stainless steel is refined iron to prevent corrosion, this is done by adding chromium (and other metals). In contrast to the first measurement, we didn't detect Molybdenum but Nickel. According to this source[3], Nickel (Ni) can also be part of the alloy for even better corrosion-resistance.

I double checked if Neodym could be fitted on the magnet's spectrum, but sadly apparently its no a Neodym-magnet. (For better evaluation it would've been better to include Neodyms enery lines with a different color, to show this clearly, sorry). The only detected element, included in a Neodym-magnet alloy, is Iron (Fe), Bor (Br) was not detected. Additionally, I detected Wolfram (W) and Titanium (Ti), two metals. The last one, Brom (Br) makes no sense, because under normal conditions, it is a liquid. More dangerously though is the fact, that Brom in elementary form is highly toxic. Nevertheless, it aligns perfectly with two peaks in the magnet's spectrum.
The unknown alloy shows different possible metals: Copper (Cu), lead (Pb), Germanium (Ge) and Arsen (As). I have no further insight on this probe, I have no idea, how likely it is for the lab to have waste pieces of those metals, to be put inside the tape just to be analyzed in this experiment. Kind of useless measurement.

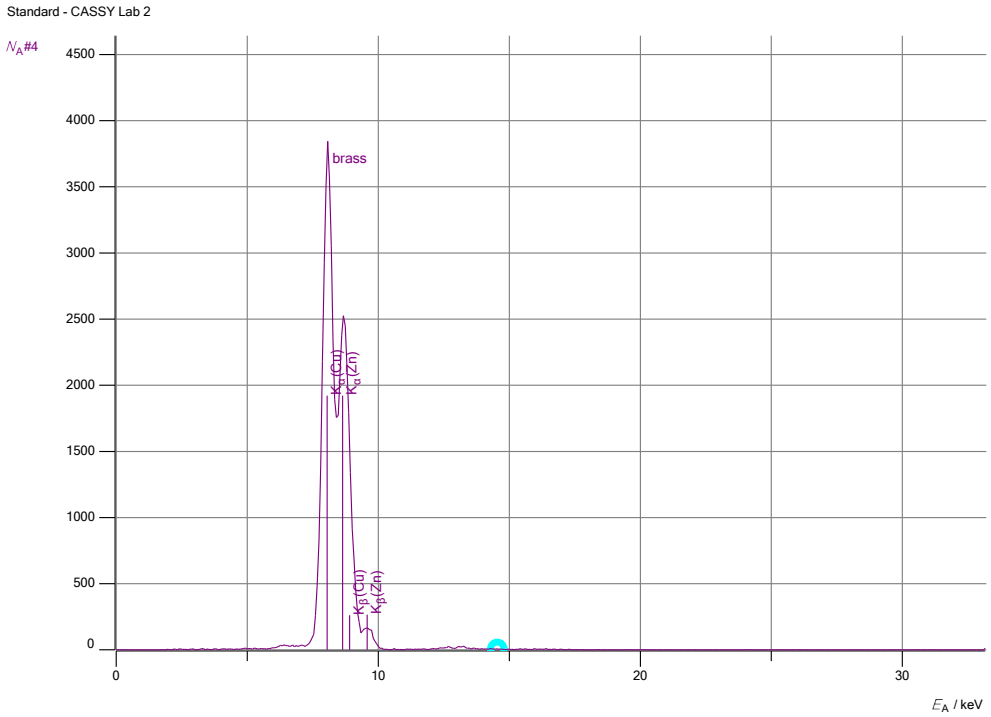


Figure 3.5: brass

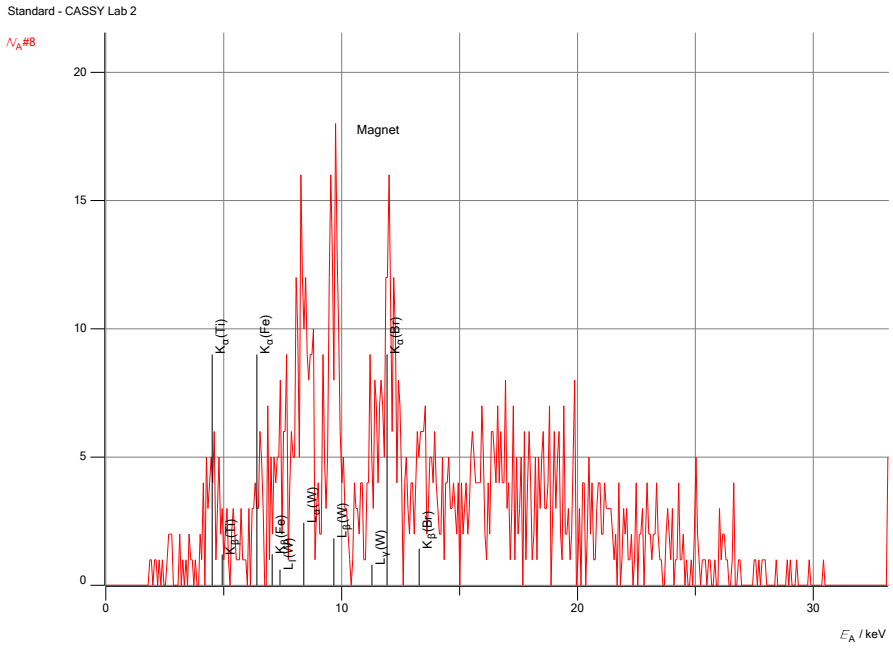


Figure 3.6: magnet

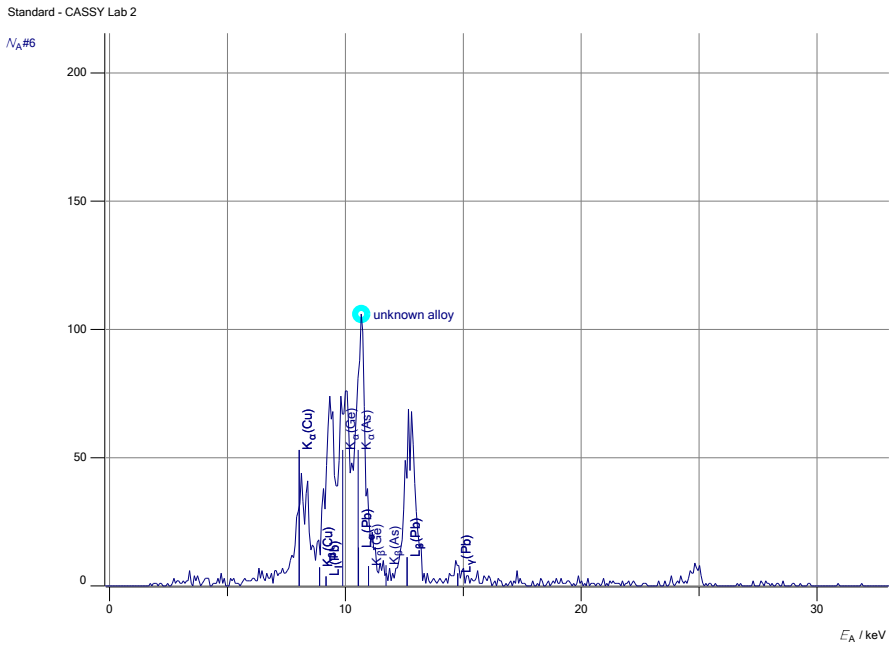


Figure 3.7: unknown-alloy

4. Discussion

The whole accuracy of the experiment relies on the calibration. So potential errors in this process/of those elements trickle down to all other measurements. As far as I understood the mechanism, I assume that only high energy radiation will trigger the detector. So any background radiation, say visible wavelengths won't trigger it. Therefore this background noise won't pose that much of an issue. The effect of dark current still exists, because it's not examined in this lab experiment, I'll view it as negligible.

E_α examination One can compare the values to the Rydberg energy's literature value:

Table 4.1: Comparing the results for E_R and σ_{12} derived by the E_α data with the literature[45]

E_R[eV]	$E_{R,\text{lit}}$[eV]	abs.Abw.[eV]	proz.Abw.[%]	$S[\sigma]$
13.80 ± 0.11	13.60514138	0.19 ± 0.11	1.4 ± 0.9	1.8
σ_{12}	$\sigma_{12,\text{lit}}$	abs.Abw.	proz.Abw.[%]	$S[\sigma]$
0.98 ± 0.15	1.0	0.02 ± 0.15	2 ± 16	0.17

We notice, that both parameters have acceptable deviations under 3σ.

E_β examination Once again for the second calculation:

Table 4.2: Comparing the results for E_R and σ_{13} derived by the E_β data with the literature[45]

E_R[eV]	$E_{R,\text{lit}}$[eV]	abs.Abw.[eV]	proz.Abw.[%]	$S[\sigma]$
13.40 ± 0.15	13.605 141 38	0.21 ± 0.15	1.5 ± 1.2	1.4
σ_{13}	$\sigma_{12,\text{lit}}$	abs.Abw.	proz.Abw.[%]	$S[\sigma]$
1.53 ± 0.22	1.8	0.27 ± 0.22	15 ± 13	1.3

I couldn't decide on a table design layout, so there are two versions here.
Once again, looking at the deviations, one draws the conclusion, that both parameters are measured in a reasonable and acceptable way.

The result errors of the second part are higher than the first one, this can be attributed to the K_β peaks: They stood out less cleary than the K_α peaks, thus the gaussian fits should be less accurate than for the main peak.

4.1. *Updated part:* Alloys

All accesible information was already discussed in the analysis. While a few elements seemingly could be identified correctly, there were also a lot of questionable results. One can argue, that there was a certain bias in fitting Cu and Zn for brass - because one knows the alloy already, one only tries to fit known elements and won't explore other elements.

Also, a lot of emission lines that were fitted on the spectrums only apply rudimentary, in german we say "über den Daumen gepeilt". Some definite peaks couldn't be fitted at all, so either the calibration is off, or the whole process has some flaws not accounted for or known of. The script's information on the process of detecting materials with this X-ray machine is scarce.

One thing, it would be useful to find out, why the height of energy lines, that can be displayed by clicking on the periodic table, differ. Some are so small, that it's very hard to accurately tell, if they correspond to a peak in the spectrum. If the setting could be found to change that, it would make things easier, but probably those different heights even have a scientific reason?

4.2. Conclusion

I can't think of nice error sources of the apparatus. What is the effect on the K_α, β lines, if the metal probes are contaminated by lets say sweat/grease/dust? What if the probes aren't pure samples, does this shift the emission line of the element? I would suggest no, because as the alloy exercise proposed, you can detect each element of an alloy/mix seperately. I don't think the energy niveaus of a single atom are shifted by positioning it near other atoms of a different kind, but I'm no expert.

Literature references

[1]Dr. J. Wagner, *Physikalisches praktikum 2.2 für studierende der physik b.sc.* [Online; Stand 04/2026] (Universität Heidelberg, 04/2026), https://git.physi.uni-heidelberg.de/schwierz/Versuchsanleitungen/src/branch/main/PAP2_2.pdf (visited on 04/14/2026) (cit. on pp. 3, 5, 11).

[2]Wikipedia, *Brass*, [Online; Stand 05/2026], https://en.wikipedia.org/wiki/Brass (cit. on p. 15).

[3]Wikipedia, *Stainless steel*, [Online; Stand 05/2026], https://de.wikipedia.org/wiki/Nichtrostender_Stahl#Beschreibung (cit. on p. 15).

[4]Wikipedia, *Rydberg unit of energy*, [Online; Stand 05/2026], https://en.wikipedia.org/wiki/Rydberg_constant (cit. on p. 18).

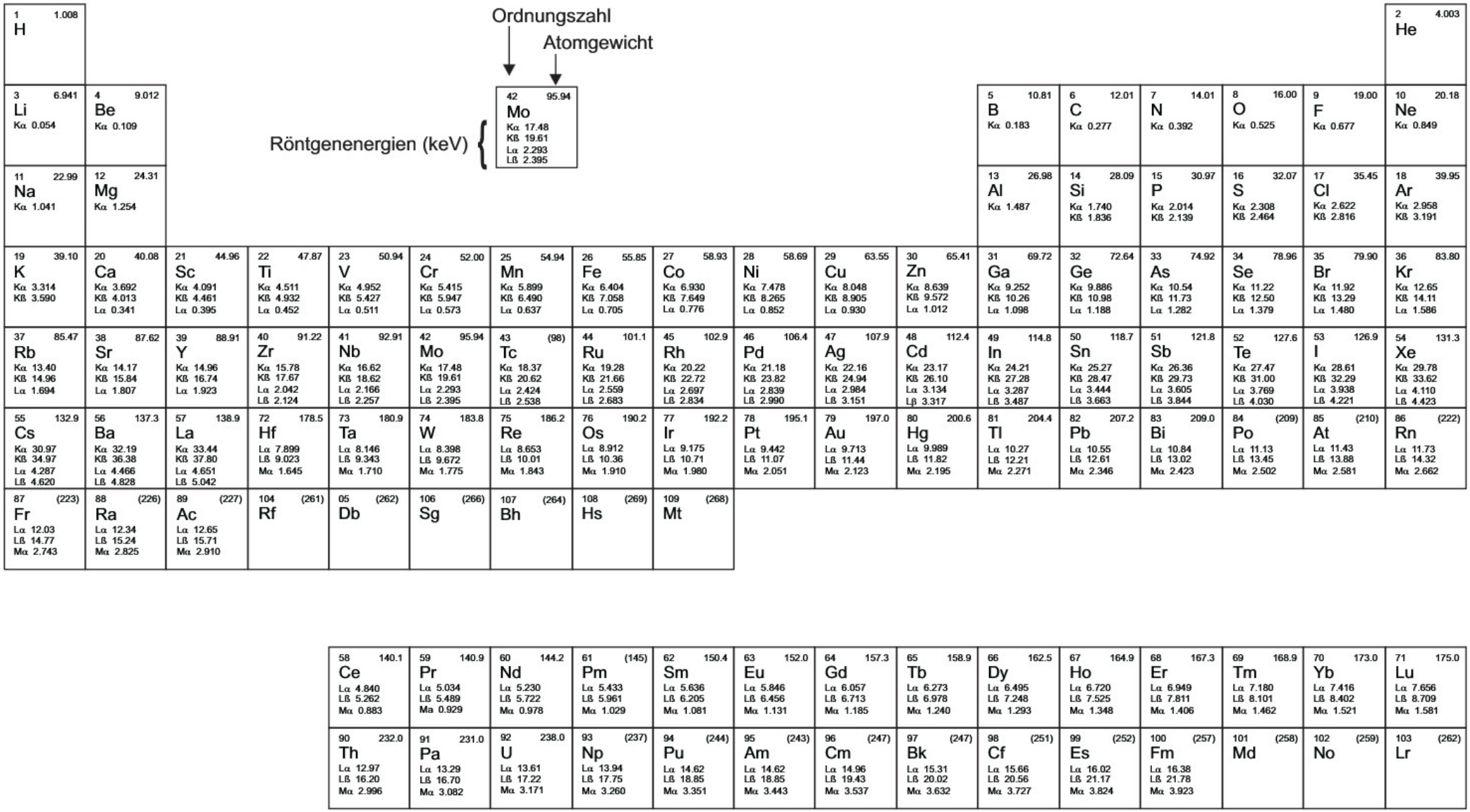
[5]Wikipedia, *Shielding constant for moseley's law*, [Online; Stand 05/2026], https://de.wikipedia.org/wiki/Moseleysches_Gesetz (cit. on p. 18).

Figure references

[6]City council of Pripyat, *Pripyat evacuation announcement*, [Online; Stand 05/2026], https://www.youtube.com/watch?v=Pl2iOG29-PI (cit. on p. 1).

[7]Wikipedia, *Iso 7010*, [Online; Stand 05/2026], https://de.wikipedia.org/wiki/ISO_7010 (cit. on pp. 1, 22).

A. Periodic table


Ordnungszahl
Atomgewicht
Röntgenenergien (keV)
42 95.94
Mo
Kα 17.48
Kß 19.61
Lα 2.293
Lß 2.395
1 1.008
H
Kα 0.054
2 4.003
He
3 6.941
Li
Kα 0.054
4 9.012
Be
Kα 0.109
5 10.81
B
Kα 0.183
6 12.01
C
Kα 0.277
7 14.01
N
Kα 0.392
8 16.00
O
Kα 0.525
9 19.00
F
Kα 0.677
10 20.18
Ne
Kα 0.849
11 22.99
Na
Kα 1.041
12 24.31
Mg
Kα 1.254
13 26.98
Al
Kα 1.487
14 28.09
Si
Kα 1.740
Kß 1.836
15 30.97
P
Kα 2.014
Kß 2.139
16 32.07
S
Kα 2.308
Kß 2.464
17 35.45
Cl
Kα 2.622
Kß 2.816
18 39.95
Ar
Kα 2.958
Kß 3.191
19 39.10
K
Kα 3.314
Kß 3.590
20 40.08
Ca
Kα 3.692
Kß 4.013
Lα 0.341
21 44.96
Sc
Kα 4.091
Kß 4.461
Lα 0.395
22 47.87
Ti
Kα 4.511
Kß 4.932
Lα 0.452
23 50.94
V
Kα 4.952
Kß 5.427
Lα 0.511
24 52.00
Cr
Kα 5.415
Kß 5.947
Lα 0.573
25 54.94
Mn
Kα 5.899
Kß 6.490
Lα 0.637
26 55.85
Fe
Kα 6.404
Kß 7.058
Lα 0.705
27 58.93
Co
Kα 6.930
Kß 7.649
Lα 0.776
28 58.69
Ni
Kα 7.478
Kß 8.265
Lα 0.852
29 63.55
Cu
Kα 8.048
Kß 8.905
Lα 0.930
30 65.41
Zn
Kα 8.639
Kß 9.572
Lα 1.012
31 69.72
Ga
Kα 9.252
Kß 10.26
Lα 1.098
32 72.64
Ge
Kα 9.886
Kß 10.98
Lα 1.188
33 74.92
As
Kα 10.54
Kß 11.73
Lα 1.282
34 78.96
Se
Kα 11.22
Kß 12.50
Lα 1.379
35 79.90
Br
Kα 11.92
Kß 13.29
Lα 1.480
36 83.80
Kr
Kα 12.65
Kß 14.11
Lα 1.586
37 85.47
Rb
Kα 13.40
Kß 14.96
Lα 1.694
38 87.62
Sr
Kα 14.17
Kß 15.84
Lα 1.807
39 88.91
Y
Kα 14.96
Kß 16.74
Lα 1.923
40 91.22
Zr
Kα 15.78
Kß 17.67
Lα 2.042
Lß 2.124
41 92.91
Nb
Kα 16.62
Kß 18.62
Lα 2.166
Lß 2.257
42 95.94
Mo
Kα 17.48
Kß 19.61
Lα 2.293
Lß 2.395
43 (98)
Tc
Kα 18.37
Kß 20.62
Lα 2.424
Lß 2.538
44 101.1
Ru
Kα 19.28
Kß 21.66
Lα 2.559
Lß 2.683
45 102.9
Rh
Kα 20.22
Kß 22.72
Lα 2.697
Lß 2.834
46 106.4
Pd
Kα 21.18
Kß 23.82
Lα 2.839
Lß 2.990
47 107.9
Ag
Kα 22.16
Kß 24.94
Lα 2.984
Lß 3.151
48 112.4
Cd
Kα 23.17
Kß 26.10
Lα 3.134
Lβ 3.317
49 114.8
In
Kα 24.21
Kß 27.28
Lα 3.287
Lß 3.487
50 118.7
Sn
Kα 25.27
Kß 28.47
Lα 3.444
Lß 3.663
51 121.8
Sb
Kα 26.36
Kß 29.73
Lα 3.605
Lß 3.844
52 127.6
Te
Kα 27.47
Kß 31.00
Lα 3.769
Lß 4.030
53 126.9
I
Kα 28.61
Kß 32.29
Lα 3.938
Lß 4.221
54 131.3
Xe
Kα 29.78
Kß 33.62
Lα 4.110
Lß 4.423
55 132.9
Cs
Kα 30.97
Kß 34.97
Lα 4.287
Lß 4.620
56 137.3
Ba
Kα 32.19
Kß 36.38
Lα 4.466
Lß 4.828
57 138.9
La
Kα 33.44
Kß 37.80
Lα 4.651
Lß 5.042
72 178.5
Hf
Lα 7.899
Lß 9.023
Mα 1.645
73 180.9
Ta
Lα 8.146
Lß 9.343
Mα 1.710
74 183.8
W
Lα 8.398
Lß 9.672
Mα 1.775
75 186.2
Re
Lα 8.653
Lß 10.01
Mα 1.843
76 190.2
Os
Lα 8.912
Lß 10.36
Mα 1.910
77 192.2
Ir
Lα 9.175
Lß 10.71
Mα 1.980
78 195.1
Pt
Lα 9.442
Lß 11.07
Mα 2.051
79 197.0
Au
Lα 9.713
Lß 11.44
Mα 2.123
80 200.6
Hg
Lα 9.989
Lß 11.82
Mα 2.195
81 204.4
Tl
Lα 10.27
Lß 12.21
Mα 2.271
82 207.2
Pb
Lα 10.55
Lß 12.61
Mα 2.346
83 209.0
Bi
Lα 10.84
Lß 13.02
Mα 2.423
84 (209)
Po
Lα 11.13
Lß 13.45
Mα 2.502
85 (210)
At
Lα 11.43
Lß 13.88
Mα 2.581
86 (222)
Rn
Lα 11.73
Lß 14.32
Mα 2.662
87 (223)
Fr
Lα 12.03
Lß 14.77
Mα 2.743
88 (226)
Ra
Lα 12.34
Lß 15.24
Mα 2.825
89 (227)
Ac
Lα 12.65
Lß 15.71
Mα 2.910
104 (261)
Rf
05 (262)
Db
106 (266)
Sg
107 (264)
Bh
108 (269)
Hs
109 (268)
Mt
58 140.1
Ce
Lα 4.840
Lß 5.262
Mα 0.883
59 140.9
Pr
Lα 5.034
Lß 5.489
Ma 0.929
60 144.2
Nd
Lα 5.230
Lß 5.722
Mα 0.978
61 (145)
Pm
Lα 5.433
Lß 5.961
Mα 1.029
62 150.4
Sm
Lα 5.636
Lß 6.205
Mα 1.081
63 152.0
Eu
Lα 5.846
Lß 6.456
Mα 1.131
64 157.3
Gd
Lα 6.057
Lß 6.713
Mα 1.185
65 158.9
Tb
Lα 6.273
Lß 6.978
Mα 1.240
66 162.5
Dy
Lα 6.495
Lß 7.248
Mα 1.293
67 164.9
Ho
Lα 6.720
Lß 7.525
Mα 1.348
68 167.3
Er
Lα 6.949
Lß 7.811
Mα 1.406
69 168.9
Tm
Lα 7.180
Lß 8.101
Mα 1.462
70 173.0
Yb
Lα 7.416
Lß 8.402
Mα 1.521
71 175.0
Lu
Lα 7.656
Lß 8.709
Mα 1.581
90 232.0
Th
Lα 12.97
Lß 16.20
Mα 2.996
91 231.0
Pa
Lα 13.29
Lß 16.70
Mα 3.082
92 238.0
U
Lα 13.61
Lß 17.22
Mα 3.171
93 (237)
Np
Lα 13.94
Lß 17.75
Mα 3.260
94 (244)
Pu
Lα 14.62
Lß 18.85
Mα 3.351
95 (243)
Am
Lα 14.62
Lß 18.85
Mα 3.443
96 (247)
Cm
Lα 14.96
Lß 19.43
Mα 3.537
97 (247)
Bk
Lα 15.31
Lß 20.02
Mα 3.632
98 (251)
Cf
Lα 15.66
Lß 20.56
Mα 3.727
99 (252)
Es
Lα 16.02
Lß 21.17
Mα 3.824
100 (257)
Fm
Lα 16.38
Lß 21.78
Mα 3.923
101 (258)
Md
102 (259)
No
103 (262)
Lr

B. Dangerous radiation symbol/Intro-Meme

The better I visualize the symbol, the less it makes sense to even display it in a python-plot. The following version is so close to the original, that I could even just save myself from those hours of work, writing the algorithm that exports path coordinates out of the inkscape image, and instead just use the original symbol.

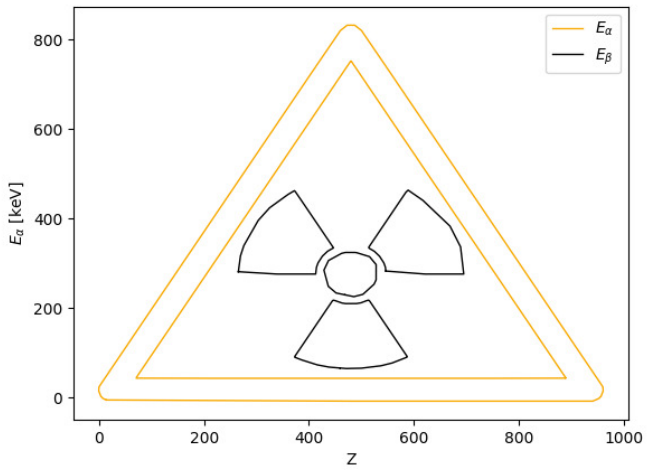


Figure B.1: reading about cchernobyl (it's forty years since the accident) is way more fun than doing linear fits[7]

C. Python-code

256

May 4, 2026

[1]:
```
# Numpy für bessere Berechnungen
import numpy as np
from numpy import exp, sqrt, log, pi
from uncertainties import unumpy as unp

# Weiteres für bessere Rechnungen
import pylab as py

from decimal import Decimal, ROUND_CEILING, ROUND_HALF_UP, getcontext #␣
 ↪Besonders für sig. Runden
import math

# Berechnungen und Plotting
from scipy import odr
import scipy.optimize
import scipy.constants as c
from scipy.stats import norm
from scipy.optimize import curve_fit
from scipy.stats import chi2
from scipy.stats import poisson
from scipy.integrate import quad
from scipy.signal import find_peaks
from scipy.signal import argrelextrema, argrelmin, argrelmax
from scipy.special import factorial

%matplotlib widget
import matplotlib.pyplot as plt
import matplotlib.mlab as mlab
import matplotlib.transforms as transforms

# Zum Auslesen von Dateien und ähnlichem
import os
import os.path
from pathlib import Path

import pandas as pd # Auch wichtig für den Latex Export
from pytexit import py2tex
```

```
import csv
import re
from typing import List


# Besseres Funktionen handling
import sympy as sp
from sympy import separatevars

# Display und Output
from IPython.display import display, Math, Latex, HTML

# Grafiken in EU-Größen ausgeben
def figsize_cm(width_cm, height_cm):
    return (2*width_cm / 2.54, 2*height_cm / 2.54)
def comma_to_float(valstr):
    return float(valstr.replace(',','.'))

g=9.80984
Delta_g=0.00002
```

[2]:
```
currentversuch = "223"
base = Path.cwd()
Messwerte_path = Path(base.parent.parent.parent/"Messwerte"/currentversuch)
 ↪ # initializing paths
Ausgabe_path = Path(base/"Diagramme")
print(Messwerte_path)
print(Ausgabe_path)
```

```
c:\Users\samue\OneDrive\Dokumente\PAP\PAP2.1\Messwerte\223
c:\Users\samue\OneDrive\Dokumente\PAP\PAP2.1\Auswertungen2.2\Code\256\Diagramme
```

Runden signifikanter Stellen

[3]:
```
def round_to_sigs(val, errVal,einheiten):
    """
    Funktion zur Rundung eines Fehlers und die Anpassung des Messwertes daran.
 ↪Diese Funktion wurde etwas umständlicher
    geschrieben, da python mit Floats und Runden schnell in Probleme rennt.
 ↪Daher musste hier mit dezimal gearbeitet werden.
    Zudem sollten besonders kleine und große Messwerte in der
 ↪Dezimalschreibweise geschrieben werden, damit diese auch
    für Protokolle geiegnet sind.

    Parameter
    ----------
    **val** : float
        Messwert
```

```
    **errVal** : float
        Ungenauigkeit des Messwertes

    Return
    ------
    **value_rounded** : str
        Gerundeter Messwert

    **error_rounded** : str
        Gerundete Ungenauigkeit des Messwertes

    **res** : str
        "Messwert \\pm Fehler"
    """

    # Daten zu Dezimal wechseln, da Floats probleme machen
    val = Decimal(str(val))
    errVal = Decimal(str(errVal))

    exp = int(math.floor(math.log10(float(errVal))))            # Die erste␣
↪signifikante Stellenposition wird anhand des errVal bestimmt

    if round(float(errVal / (Decimal(10) ** exp))) < 3:         # wenn 1,2,3␣
↪erste signifikante Stelle, dann kommt eine zweite Nachkommastellenposition␣
↪hinzu
        exp -= 1

    scale = Decimal(10) ** exp

    val_round = (val / scale).quantize(Decimal('1'), rounding=ROUND_HALF_UP) *␣
↪scale            # mit scale wird das Komma so verschoben, dass alle␣
↪signifikanten Stellen vor dem Komma stehen, und dann alle Nachkommastellen␣
↪weggerundet werden
    err_round = (errVal / scale).quantize(Decimal('1'), rounding=ROUND_CEILING)␣
↪* scale

    qty = f"\\qty{{{val_round}({err_round})}}{{{einheiten}}}"
    num = f"\\num{{{val_round}({err_round})}}"
    return float(val_round), float(err_round), num, qty

v_round = np.vectorize(round_to_sigs, otypes=[float, float, object, object],␣
↪excluded=['einheiten'])

    # wissenschaftliche Notation erstmal vernachlässigen (da häufig Einheiten␣
↪gewollt sind, die 10^/e Prefixe beinhalten)
```

Gausssche Fehlerfortpflanzung

[4]:
```
def gff(function, errPronePar,latex=True):
    """
    Kann die Fehlerformel einer gegebenen Gleichung bestimmen.

    Parameters
    ----------
    **func** : sympy function
        Funktion dessen Fehler bestimmt werden soll.

    **errPronePar** : Array
        Liste (Array) aller fehlerbehafteten Größen der Gleichung con sp.Symbols
        Diese Werte werden als x_sym, y_sym, z_sym etc. bezeichnet und sind␣
↪ungleich den Werten für x, y, z.
        Für die Werte wird daher die Bezeichnung x_val, y_val, z_val etc.␣
↪genutzt und für deren Fehler err_x, err_y, err_z etc.

    Return
    ----------
    **absolut_err** : sympy function
        Gibt die Fehlergleichung des absoluten Fehlers wieder.

    **relativ_err** : sympy function
        Gibt die Fehlergleichung des relativen Fehlers wieder.

    **errProneParamters** : array
        Liste aller Fehlerbehafteten Größen
    """

    error = 0
    errProneParamaters = []

    function = sp.sympify(function)
    variables = errPronePar.split(",")
    variables = sp.symbols(variables)

    for variable in variables:
        Delta = sp.Symbol(f"Delta_{variable}")
        partial = sp.diff(function, variable)                    # Die Funktion␣
↪wird nach der fehlerbehafteten Variable abgeleitet
        term=sp.UnevaluatedExpr(partial*Delta)**2
        error = error + ((term))                                 # Fehler werden␣
↪quadratisch aufsummiert
        errProneParamaters.append((variable,Delta))

    absolute_err=sp.simplify(sp.sqrt(error),rational = True)
```

```
    relative_err=sp.simplify(sp.sqrt(error/function**2),rational = True)

    if latex:
        #Prints out the Latex Code for the Functions
        print("Funktion:")
        display(Math(sp.latex(function,long_frac_ratio=2)))
        print(sp.latex(function))
        print("Absoluter Fehler:")
        display(Math(sp.latex(absolute_err,long_frac_ratio=2)))
        print(sp.latex(absolute_err))
        print("Relativer Fehler:")
        display(Math(sp.latex(relative_err,long_frac_ratio=2)))
        print(sp.latex(relative_err))
    return function,absolute_err, relative_err, errProneParamaters
```

Sigma-Abweichungen

[5]:
```
def sigma_abweichung(p1, err_p1,p2,err_p2):
    """
    Funktion zum berechnen der Sigma-Abweichugn von zwei Messwerten, oder einem␣
↪Messwert und einem Literaturwert.
    """
    if err_p1 == 0 and err_p2 == 0:
        raise ValueError("Für die Sigma-Abweichung muss mindestens ein Wert␣
↪fehlerbehaftet sein!")
    else:
        abweichung=Decimal(str(abs(p1 - p2)/(np.sqrt(err_p1 ** 2+err_p2 ** 2))))

    if abweichung == 0:
        return 0.0, "\\num{0}"

    exp = int(math.floor(math.log10(float(abweichung))))
    if round(float(abweichung / (Decimal(10) ** exp))) < 3:          # wenn␣
↪1,2,3 erste signifikante Stelle, dann kommt eine zweite␣
↪Nachkommastellenposition hinzu
        exp -= 1
    scale = Decimal(10) ** exp
    abweichung_round = (abweichung / scale).quantize(Decimal('1'),␣
↪rounding=ROUND_CEILING) * scale
    res = f"\\num{{{abweichung_round}}}"

    return float(abweichung_round),res
```

[6]:
```
# #Größenvergleich in latex
# def size_comp_str(name1,name2,val1,val2):
#     if val1<val2:
#         return name1+"<"+name2
```

```
#     elif val1>val2:
#         return name1+">"+name2
#     else:
#         return name1+"="+name2

#Erstellung von Vergleichstabellen in Latex
import textwrap
def compare_table(nam1,nam2,einheiten,val1,err1,val2,err2):
    VAL1=round_to_sigs(val1,err1,r'')[2]
    VAL2=round_to_sigs(val2,err2,r'')[2]
    abs=np.abs(val1-val2)
    abs_delta=np.sqrt(err1**2+err2**2)
    if abs_delta!=0:
        SIGMA=sigma_abweichung(val1,err1,val2,err2)[1]
    else:
        SIGMA=0.0
    ABS=round_to_sigs(abs,abs_delta,r'')[2]
    rel=abs/np.abs(val1)*100
    rel_delta=rel*np.sqrt((abs_delta/abs)**2+(err1/val1)**2) if abs != 0 else 0
    REL=round_to_sigs(rel,rel_delta,r'')[2]
    output = textwrap.dedent(f"""
        \\begin{{table}}[htb]
        \\centering
        \\caption{{}}
        \\begin{{tabularx}}{{0.8\\textwidth}}{{ZZZZZ}}
            \\toprule
                ␣
 ↪{nam1}[\\unit{{{einheiten}}}]&{nam2}[\\unit{{{einheiten}}}]&\\text{{abs.Abw.
 ↪}}[\\unit{{{einheiten}}}]&\\text{{proz.Abw.
 ↪}}[\\unit{{\\percent}}]&S[\\sigma]\\\\\\midrule
                {VAL1}&{VAL2}&{ABS}&{REL}&{SIGMA}\\\\
            \\bottomrule
        \\end{{tabularx}}
        \\label{{table:Vergleich_{nam1}}}
        \\end{{table}}
    """)
    print(output)
```

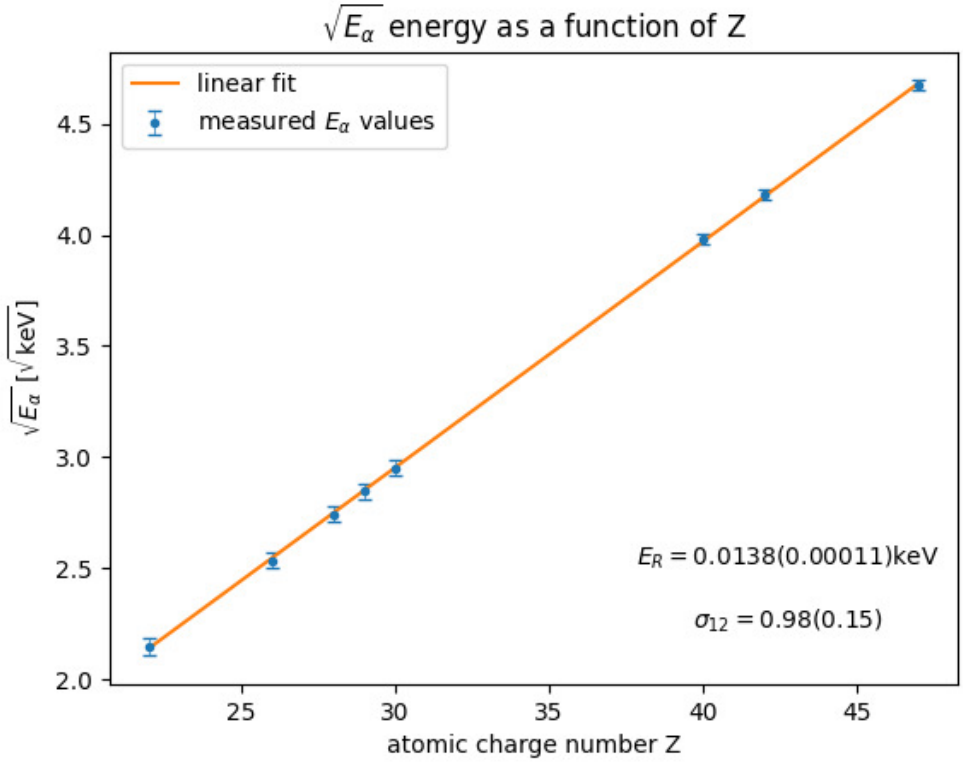
[7]:
```
# #Größenvergleich in latex
# def size_comp_str(name1,name2,val1,val2):
#     if val1<val2:
#         return name1+"<"+name2
#     elif val1>val2:
#         return name1+">"+name2
#     else:
#         return name1+"="+name2
```

```
#Erstellung von Vergleichstabellen in Latex
def compare_table_withliterature(nam1,nam2,einheiten,val1,err1,val2):
    VAL1=round_to_sigs(val1,err1,r'')[2]
    abs=np.abs(val1-val2)
    abs_delta=np.sqrt(err1**2)
    if abs_delta!=0:
        SIGMA=sigma_abweichung(val1,err1,val2,0)[1]
    else:
        SIGMA=0.0
    ABS=round_to_sigs(abs,abs_delta,r'')[2]
    rel=abs/np.abs(val2)*100
    rel_delta=rel*np.sqrt((abs_delta/abs)**2+(err1/val1)**2) if abs != 0 else 0
    REL=round_to_sigs(rel,rel_delta,r'')[2]
    output = textwrap.dedent(f"""
        \\begin{{table}}[htb]
        \\centering
        \\caption{{}}
        \\begin{{tabularx}}{{0.8\\textwidth}}{{ZZZZZ}}
            \\toprule
                ␣
 ↪{nam1}[\\unit{{{einheiten}}}]&{nam2}[\\unit{{{einheiten}}}]&\\text{{abs.Abw.
 ↪}}[\\unit{{{einheiten}}}]&\\text{{proz.Abw.
 ↪}}[\\unit{{\\percent}}]&S[\\sigma]\\\\\\midrule
                {VAL1}&\\num{{{val2}}}&{ABS}&{REL}&{SIGMA}\\\\
            \\bottomrule
        \\end{{tabularx}}
        \\label{{table:Vergleich_{nam1}}}
        \\end{{table}}
    """)
    print(output)
```

[8]:
```
Z=np.array([22,26,28,29,30,40,42,47])
#K_alpha (Ti, Fe, Ni, Cu, Zn, Zr, Mo, Ag) in keV:
K_alpha=np.array([4.61,6.44,7.52,8.10,8.72,15.84,17.49,21.84])
Delta_K_alpha=np.array([0.17,0.18,0.19,0.19,0.19,0.18,0.19,0.19])
sqrt_K_alpha=np.sqrt(K_alpha)
Delta_sqrt_K_alpha=Delta_K_alpha/2/np.sqrt(K_alpha)

n1=1
n2=2
def fit_func(x, sqrt_ER, sig12):
    return sqrt_ER*(x-sig12)*np.sqrt(1/n1**2-1/n2**2)
popt, pcov=curve_fit(fit_func, Z,sqrt_K_alpha,sigma=Delta_sqrt_K_alpha)
E_R,Delta_E_R,_,latexer=round_to_sigs(popt[0]**2,2*popt[0]*np.
 ↪sqrt(pcov[0][0]),r'\kilo\electronvolt')
sigma,Delta_sigma,_,latexsigma=round_to_sigs(popt[1],np.sqrt(pcov[1][1]),r'')
```

```
plt.close('all')
plt.errorbar(Z, sqrt_K_alpha, Delta_sqrt_K_alpha, fmt="."
  ↪",capsize=3,elinewidth=0.5,label=r'measured $E_\alpha$ values',)
plt.plot(Z, fit_func(Z,*popt),label='linear fit')
plt.xlabel(r'atomic charge number Z')
plt.ylabel(r'$\sqrt{E_\alpha}$ [$\sqrt{\mathrm{keV}}$]')
plt.title(r'$\sqrt{E_\alpha}$ energy as a function of Z')
plt.text(0.8, 0.2,rf'$E_R={E_R}({Delta_E_R})$keV',␣
  ↪horizontalalignment='center',verticalalignment='center', transform=plt.gca().
  ↪transAxes)
plt.text(0.8, 0.1,rf'$\sigma_{{12}}={sigma}({Delta_sigma})$',␣
  ↪horizontalalignment='center',verticalalignment='center', transform=plt.gca().
  ↪transAxes)
print(latexer)
print(E_R)
print(latexsigma)
plt.legend()
plt.show()
plt.savefig(Ausgabe_path/"Nr.1_Kalpha.png",format="png")
```

```
\qty{0.01380(0.00011)}{\kilo\electronvolt}
0.0138
\qty{0.98(0.15)}{}
```



[9]:
```
# compare_table_withliterature(nam1,nam2,einheiten,val1,err1,val2):
compare_table_withliterature(r'E_R',r'E_{R,\text{lit}}',r'\electronvolt',13.8,0.
↪11,13.605141380)
```

```
\begin{table}[htb]
\centering
\caption{}
\begin{tabularx}{0.8\textwidth}{ZZZZZ}
    \toprule
        E_R[\unit{\electronvolt}]&E_{R,\text{lit}}[\unit{\electronvolt}]&\text{a
bs.Abw.}[\unit{\electronvolt}]&\text{proz.Abw.}[\unit{\percent}]&S[\sigma]\\\mid
rule
\num{13.80(0.11)}&\num{13.60514138}&\num{0.19(0.11)}&\num{1.4(0.9)}&\num{1.8}\\
    \bottomrule
\end{tabularx}
\label{table:Vergleich_E_R}
\end{table}
```

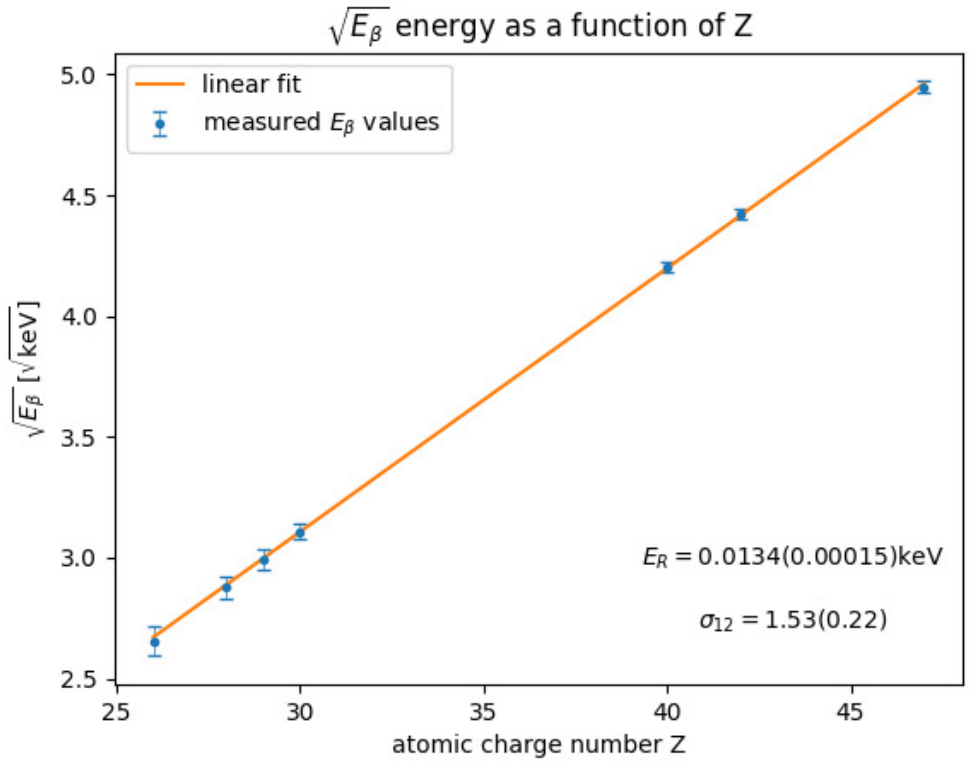
[10]:
```
compare_table_withliterature(r'\sigma_{12}',r'\sigma_{12,\text{lit}}',r'',0.
↪9751,0.15,1.0)
```

```
\begin{table}[htb]
\centering
\caption{}
\begin{tabularx}{0.8\textwidth}{ZZZZZ}
    \toprule
        \sigma_{12}[\unit{}]&\sigma_{12,\text{lit}}[\unit{}]&\text{abs.Abw.}[\un
it{}]&\text{proz.Abw.}[\unit{\percent}]&S[\sigma]\\\midrule
        \num{0.98(0.15)}&\num{1.0}&\num{0.02(0.15)}&\num{2(16)}&\num{0.17}\\
    \bottomrule
\end{tabularx}
\label{table:Vergleich_\sigma_{12}}
\end{table}
```

[11]:
```
#K_beta (Fe, Ni, Cu, Zn, Zr, Mo, Ag) in keV:
K_beta=np.array([7.04,8.28,8.97,9.66,17.69,19.56,24.49])
Delta_K_beta=np.array([0.32,0.27,0.25,0.19,0.18,0.16,0.23])
sqrt_K_beta=np.sqrt(K_beta)
Delta_sqrt_K_beta=Delta_K_beta/2/np.sqrt(K_beta)

n1=1
```

```
n2=3
def fit_func(x, sqrt_ER, sig12):
    return sqrt_ER*(x-sig12)*np.sqrt(1/n1**2-1/n2**2)
popt, pcov=curve_fit(fit_func,Z[1:],sqrt_K_beta,sigma=Delta_sqrt_K_beta)
E_R,Delta_E_R,_,latexer=round_to_sigs(popt[0]**2,2*popt[0]*np.
 ↪sqrt(pcov[0][0]),r'\kilo\electronvolt')
sigma,Delta_sigma,_,latexsigma=round_to_sigs(popt[1],np.sqrt(pcov[1][1]),r'')

plt.close('all')
plt.errorbar(Z[1:], sqrt_K_beta, Delta_sqrt_K_beta, fmt="."
 ↪",capsize=3,elinewidth=0.5,label=r'measured $E_\beta$ values',)
plt.plot(Z[1:], fit_func(Z[1:],*popt),label='linear fit')
plt.xlabel(r'atomic charge number Z')
plt.ylabel(r'$\sqrt{E_\beta}$ [$\sqrt{\mathrm{keV}}$]')
plt.title(r'$\sqrt{E_\beta}$ energy as a function of Z')
plt.text(0.8, 0.2,rf'$E_R={E_R}({Delta_E_R})$keV',␣
 ↪horizontalalignment='center',verticalalignment='center', transform=plt.gca().
 ↪transAxes)
plt.text(0.8, 0.1,rf'$\sigma_{{12}}={sigma}({Delta_sigma})$',␣
 ↪horizontalalignment='center',verticalalignment='center', transform=plt.gca().
 ↪transAxes)
print(latexer)
print(latexsigma)
plt.legend()
plt.show()
plt.savefig(Ausgabe_path/"Nr.1_Kbeta.png",format="png")
```

```
\qty{0.01340(0.00015)}{\kilo\electronvolt}
\qty{1.53(0.22)}{}
```



[12]:
```
compare_table_withliterature(r'E_R',r'E_{R,\text{lit}}',r'\electronvolt',13.
↪40,0.15,13.605141380)

compare_table_withliterature(r'\sigma_{13}',r'\sigma_{12,\text{lit}}',r'',1.
↪53,0.22,1.8)
```

```
\begin{table}[htb]
\centering
\caption{}
\begin{tabularx}{0.8\textwidth}{ZZZZZ}
    \toprule
        E_R[\unit{\electronvolt}]&E_{R,\text{lit}}[\unit{\electronvolt}]&\text{a
bs.Abw.}[\unit{\electronvolt}]&\text{proz.Abw.}[\unit{\percent}]&S[\sigma]\\\mid
rule
\num{13.40(0.15)}&\num{13.60514138}&\num{0.21(0.15)}&\num{1.5(1.2)}&\num{1.4}\\
    \bottomrule
\end{tabularx}
\label{table:Vergleich_E_R}
\end{table}
```

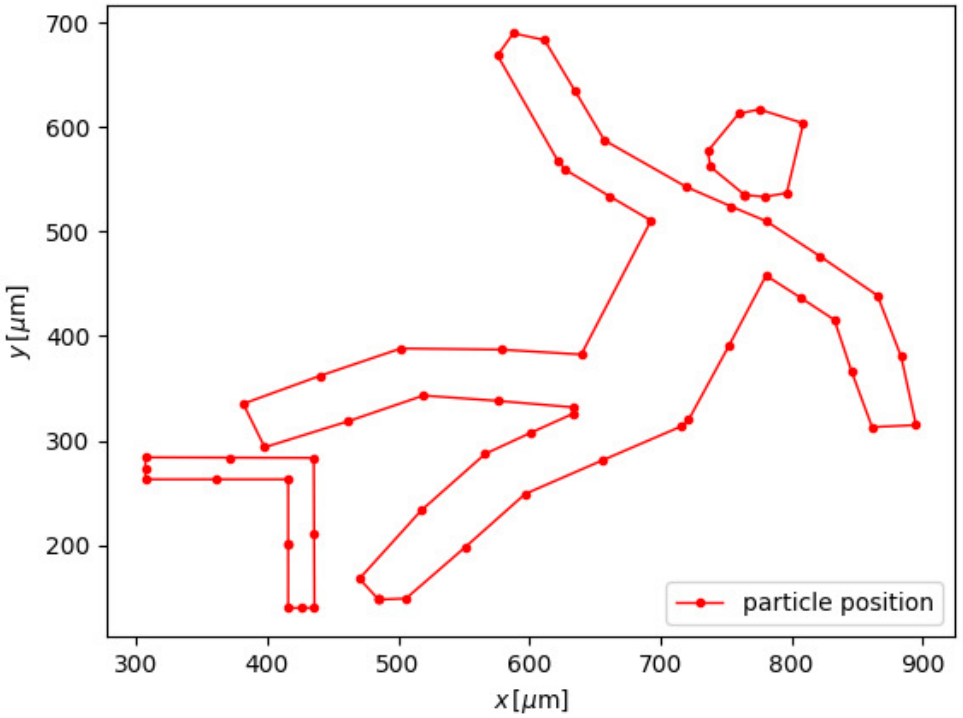
```
\begin{table}[htb]
\centering
\caption{}
\begin{tabularx}{0.8\textwidth}{ZZZZZ}
    \toprule
        \sigma_{13}[\unit{}]&\sigma_{12,\text{lit}}[\unit{}]&\text{abs.Abw.}[\un
it{}]&\text{proz.Abw.}[\unit{\percent}]&S[\sigma]\\\midrule
        \num{1.53(0.22)}&\num{1.8}&\num{0.27(0.22)}&\num{15(13)}&\num{1.3}\\
    \bottomrule
\end{tabularx}
\label{table:Vergleich_\sigma_{13}}
\end{table}
```

[13]:
```
data1="""
# M 416 921.5
# L 416 860
# L 361.5 860
# L 307 860
# L 307 849.49
# L 307 838.99
# L 371.25 839.24
# L 435.5 839.5
# L 435.76 911.25
# L 436.01 983
# L 426.01 983
# L 416 983
# Z

"""
data2="""
M 485 974.98
C 476.49 971.01 471.39 964.09 470.32 955.06
C 469.11 944.83 469.08 944.88 517.45 889.81
C 541.88 862 563.87 837.61 566.31 835.61
C 568.75 833.6 584.42 824.51 601.12 815.39
C 617.83 806.28 632.29 798.12 633.25 797.26
C 635.34 795.39 635.52 792.67 633.65 791.12
C 632.87 790.48 608.39 787.86 575.75 784.93
L 519.19 779.86
L 461.42 804.6
C 408.88 827.1 403.15 829.33 398.08 829.32
C 376.59 829.28 366.09 802.58 381.73 787.74
C 383.64 785.92 403.85 776.69 440.57 760.87
C 471.33 747.61 498.98 735.89 502 734.82
C 512.93 730.95 514.47 730.97 578.5 735.92
```

```
C 612.05 738.51 639.53 740.6 639.57 740.57
C 640.05 740.14 693.03 612.71 692.81 612.52
C 692.64 612.38 678.55 602.04 661.5 589.55
C 644.45 577.05 629.22 565.63 627.65 564.16
C 626.09 562.7 623.34 558.58 621.54 555
C 613.39 538.77 576.16 457.19 575.71 454.57
C 574.39 446.92 580.25 436.58 587.72 433.38
C 595.65 429.98 607.05 433.09 611.52 439.87
C 612.84 441.87 623.4 463.95 635 488.95
C 646.6 513.95 656.62 535.04 657.26 535.82
C 658.71 537.56 709.84 574.02 719.5 580.19
C 723.35 582.65 738.65 591.03 753.5 598.82
L 780.5 612.97
L 821.21 646.26
C 843.6 664.58 863.54 681.64 865.52 684.19
C 868.89 688.51 870.11 692.46 883.6 742.66
C 899.09 800.27 899.05 800.06 894.95 808.09
C 888.77 820.21 868.91 821.37 861.42 810.05
C 859.7 807.46 855.02 791.54 845.81 756.92
L 832.65 707.5
L 806.75 686.15
C 792.51 674.41 780.71 664.96 780.52 665.15
C 780.34 665.34 767.57 695.59 752.14 732.37
C 736.72 769.15 722.9 800.98 721.44 803.09
C 719.97 805.21 717.16 808.16 715.2 809.66
C 713.23 811.16 686.62 825.54 656.06 841.61
C 625.5 857.68 598.83 872.24 596.78 873.95
C 594.73 875.66 574.35 898.36 551.48 924.38
C 523.23 956.51 508.6 972.42 505.84 973.97
C 499.96 977.27 490.86 977.72 485 974.98
Z
"""
data3="""
M 764.27 588.1
C 751.75 583.46 742.28 573.67 737.96 560.92
C 736.22 555.76 735.87 552.83 736.19 545.98
C 736.93 530.25 745.43 517.36 759.72 510.29
C 765.99 507.2 768.39 506.57 775.4 506.2
C 789.36 505.47 799.42 509.47 808.93 519.52
C 828.35 540.06 822.03 573.5 796.34 586.15
C 790.29 589.13 788.61 589.49 779.56 589.75
C 771.56 589.98 768.43 589.64 764.27 588.1
Z
"""
data = [data1, data2, data3]
```

```
pattern = r'([MLCZ])\s*([\d\s.-]+)'      # ([]) ist Gruppe, speichert in␣
 ↪Gruppe1 den Buchstaben [M oder L oder C] ab     \s* beliebig viele␣
 ↪Leerzeichen  ()Gruppe zwei

plt.close('all')

for i in range(3):
    x=[]
    y=[]
    for command, values in re.findall(pattern, data[i]):
        # Alle Zahlen im Block extrahieren und in Floats umwandeln
        nums = [float(n) for n in re.findall(r'[-+]?\d*\.?\d+', values)] #[-+]?:
 ↪ negative Zahlen       \d*: beliebig viele Ziffern      \.?: optionaler␣
 ↪Dezimalpunkt    \d+: Sucht nach mindestens einer Ziffer nach dem Punkt und␣
 ↪wird von leerzeichen gestoppt

        if command == 'M' or command == 'L':
            # Nimm das erste (und einzige) Paar: [x, y]
            x.append(nums[0])
            y.append(nums[1])


        elif command == 'C':
            # Ein C-Befehl hat 6 Zahlen (x1, y1, x2, y2, x3, y3)
            # Wir wollen nur das letzte Paar (Index 4 und 5)
            x.append(nums[4])
            y.append(nums[5])

        elif command =='Z':
            x.append(x[0])      # Startpunkt = Endpunkt
            y.append(y[0])

    x=np.array(x)
    y=1123-np.array(y)
    if i == 0:
        # Nur beim ersten Durchgang das Label setzen
        plt.plot(x, y, marker='.', color='red', linewidth=1, label='particle␣
 ↪position')
    else:
        # Danach ohne Label plotten
        plt.plot(x, y, marker='.', color='red', linewidth=1)
# plt.title('Deterministisches Leiden eines Studierenden-Partikels')
plt.xlabel(r'$x\,[\mu$m]')
plt.ylabel(r'$y\,[\mu$m]')
plt.legend(loc='lower right')

plt.show()
```

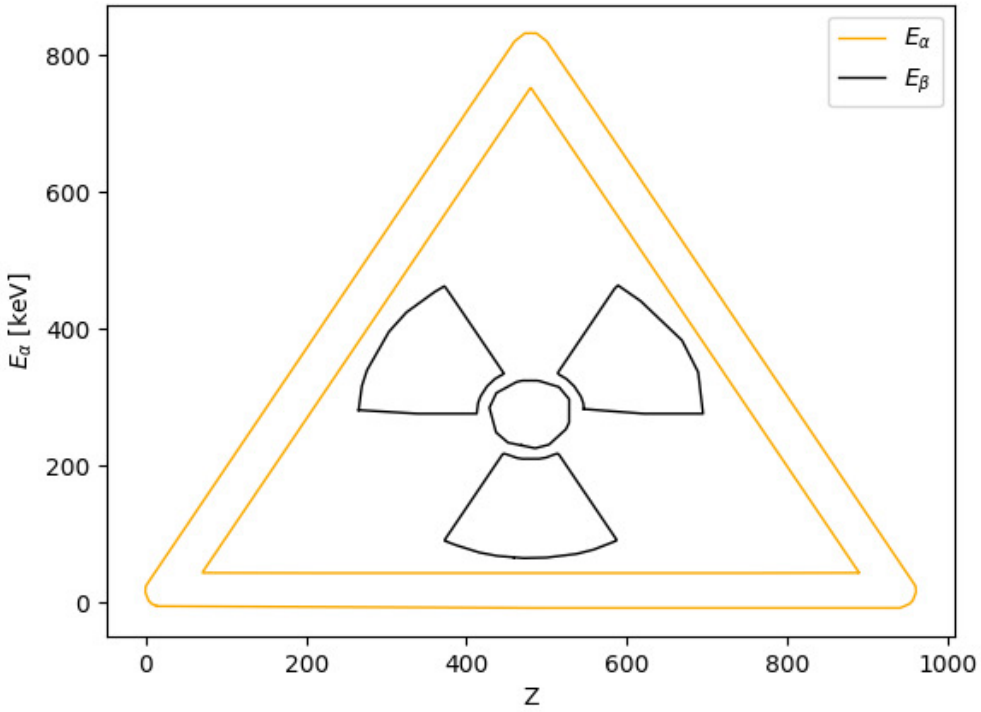
```
plt.savefig(Ausgabe_path/"Meme.png",format="png")


# pattern = r'\d*\.?\d'
# coordinates=re.findall(pattern, data)
# x=[]
# y=[]
# for i in range(0, len(coordinates), 2):
#     x.append(coordinates[i])
#     y.append(coordinates[i+1])


# for line in data.strip().splitlines():
#     matches = re.findall(pattern, line)
#     if len(matches) == 2:
#         xvalue = map(float, matches[0])
#         yvalue = map(float, matches[1])
#         x.append(xvalue)
#         y.append(yvalue)
```



[19]:
```
data="""
M 14.19,835.41
C 7.94,832.29 3.59,827.28 1.38,820.64 -0.51,814.95 -0.05,806.94 2.44,802.12 3.
↪32,800.41 106.42,621.6 231.55,404.76 406.89,100.91 459.98,9.66 463.07,6.83␣
↪472.79,-2.07 487.21,-2.07 496.94,6.83 500.03,9.66 553.06,100.82 728.51,404.
↪86 853.66,621.76 956.76,800.58 957.62,802.23 960.06,806.95 960.5,814.99 958.
↪62,820.64 956.41,827.28 952.06,832.29 945.81,835.41
L 940.61,838
H 480 19.39
Z

M 888.83,786.03
C 888.14,783.99 480.48,78.05 480,78.06 479.53,78.06 71.88,784 71.17,786.05 70.
↪96,786.65 220.55,787 480,787 745.21,787 889.05,786.66 888.83,786.03
Z

M 459.14,763.97
C 438.41,761.96 415.79,756.9 398,750.29 386.1,745.87 373,739.95 373,738.99␣
↪373,738.14 444.62,613.98 445.78,612.82 446.21,612.39 449.71,613.32 453.
↪57,614.89 463.16,618.8 469.49,620 480.3,619.99 490.65,619.97 499.03,618.29␣
↪507.56,614.52 510.79,613.09 513.73,612.33 514.21,612.81 515.43,614.03␣
↪587,738.07 587,738.97 587,739.37 583.96,741.09 580.25,742.79 558.3,752.82␣
↪537.23,758.96 512.5,762.55 499.8,764.39 471.32,765.15 459.14,763.97
Z

M 468.63,600.41
C 451.75,596.33 436.88,581.42 433.02,564.72 428.54,545.28 437.25,523.75 453.
↪93,513.05 462.22,507.73 469.37,505.72 480,505.72 487.21,505.72 490.93,506.23␣
↪495.42,507.86 515.32,515.08 528,532.86 527.96,553.5 527.94,566.9 524.09,576.
↪63 514.92,586.43 502.87,599.31 485.61,604.52 468.63,600.41
Z

M 265.53,548.75
C 269.51,513.61 275.97,490.4 289.55,462.5 303.35,434.15 325.23,405.73 348.
↪28,386.2 360.54,375.82 372.16,367.76 373.27,368.87 373.78,369.38 390.54,398.
↪08 410.5,432.64
L 446.78,495.47 442.13,498.6
C 435.51,503.06 425.73,513.86 421.66,521.21 417.4,528.9 414.45,538.32 413.
↪49,547.27
L 412.77,554
H 338.85 264.93
Z

M 546.51,547.27
```

```
C 545.55,538.32 542.6,528.9 538.34,521.21 534.39,514.07 524.7,503.26 518.3,498.
↪87 515.94,497.24 514,495.5 514,495.01 514,493.93 584.79,371.04 586.46,369.21␣
↪588.96,366.49 617.14,389.37 632.82,406.86 668.71,446.89 688.81,493.22 694.
↪38,548.75
L 694.91,554
H 621.07 547.23
Z
"""
data_blocks = [b.strip() for b in data.split('\n\n') if b.strip()]

pattern = r'([MLCZHV])\s*([\d\s.,-]+)?'      # ([]) ist Gruppe, speichert in␣
↪Gruppe1 den Buchstaben [M oder L oder C] ab     \s* beliebig viele␣
↪Leerzeichen  ()Gruppe zwei

plt.close('all')

for i, block in enumerate(data_blocks):
    x=[]
    y=[]
    for command, values in re.findall(pattern, block):
        # Alle Zahlen im Block extrahieren und in Floats umwandeln
        nums = [float(n) for n in re.findall(r'[-+]?\d*\.?\d+', values.
↪replace(',', ' '))] #[-+]?: negative Zahlen       \d*: beliebig viele␣
↪Ziffern      \.?: optionaler Dezimalpunkt    \d+: Sucht nach mindestens␣
↪einer Ziffer nach dem Punkt und wird von leerzeichen gestoppt

        # elif command == 'C' or command =='c':
        #     # Ein C-Befehl hat 6 Zahlen (x1, y1, x2, y2, x3, y3)
        #     # Wir wollen nur das letzte Paar (Index 4 und 5)
        #     x.append(nums[-2])
        #     y.append(nums[-1])

        if command == 'M' or command == 'L':
            if len(nums) >= 2:
                last_x, last_y = nums[0], nums[1]
                x.append(last_x)
                y.append(last_y)

        elif command == 'C':
        # Wir gehen in 6er-Schritten durch die Zahlen (i = 0, 6, 12...)
            for j in range(0, len(nums), 2):
            # Wir prüfen, ob wir genug Zahlen für ein komplettes Segment haben
                # Der echte Knoten ist immer das letzte Paar einer 6er-Gruppe
                    x.append(nums[j])
                    y.append(nums[j+1])

            # for j in range(0, len(nums), 6):
```

```
            # # Wir prüfen, ob wir genug Zahlen für ein komplettes Segment haben
            #     if j + 5 < len(nums):
            #     # Der echte Knoten ist immer das letzte Paar einer 6er-Gruppe
            #         x.append(nums[j+4])
            #         y.append(nums[j+5])
        elif command == 'H':
            # Nur X ändert sich, Y bleibt gleich
            if len(nums) >= 1:
                last_x = nums[0]
                x.append(last_x)
                y.append(last_y)

        elif command == 'V':
            # Nur Y ändert sich, X bleibt gleich
            if len(nums) >= 1:
                last_y = nums[0]
                x.append(last_x)
                y.append(last_y)

        elif command == 'Z':
            if x:
                x.append(x[0])
                y.append(y[0])
                last_x, last_y = x[0], y[0]

    x=np.array(x)
    y=830-np.array(y)
    if i == 0:
        labeltext=r'$E_\alpha$'
    elif i==2:
        labeltext=r'$E_\beta$'        # Nur beim nullten Durchgang das Label␣
 ↪setzen
    else: labeltext=None

    if i>1:
        plt.plot(x, y,color=f"black",linewidth=1, label=labeltext)#color=f"C{1}"
    else:
        plt.plot(x, y,color='#f9aa00',linewidth=1,␣
 ↪label=labeltext)#color=f"C{1}"

plt.xlabel(r'Z')
plt.ylabel(r'$E_\alpha$ [keV]')
plt.legend()

plt.show()
plt.savefig(Ausgabe_path/"Meme.png",format="png")
```

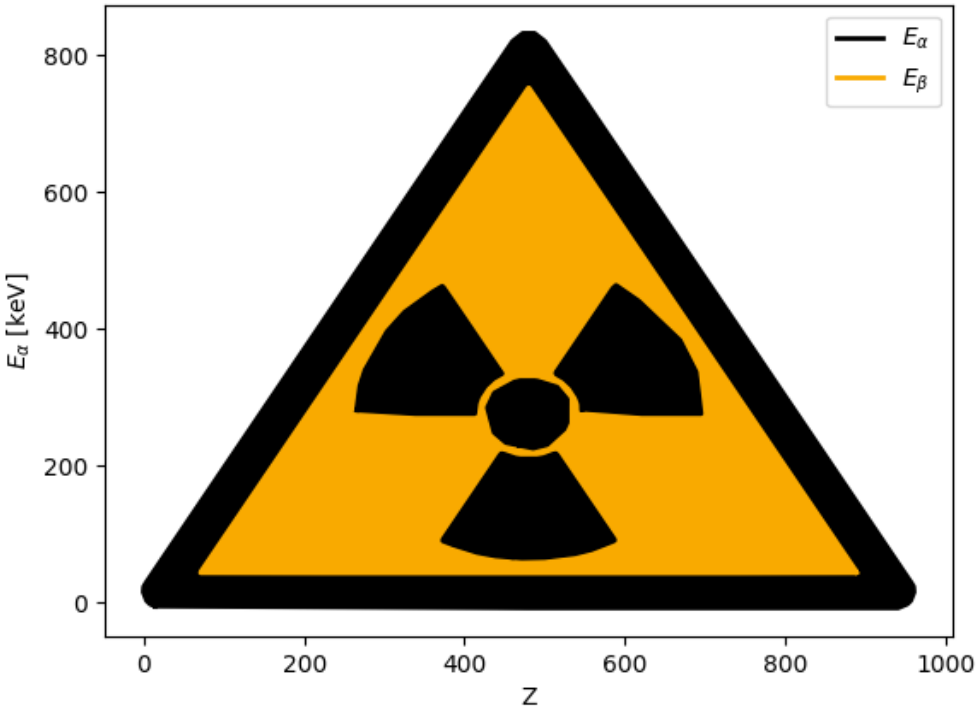


[18]:
```
data="""
M 14.19,835.41
C 7.94,832.29 3.59,827.28 1.38,820.64 -0.51,814.95 -0.05,806.94 2.44,802.12 3.
↪32,800.41 106.42,621.6 231.55,404.76 406.89,100.91 459.98,9.66 463.07,6.83␣
↪472.79,-2.07 487.21,-2.07 496.94,6.83 500.03,9.66 553.06,100.82 728.51,404.
↪86 853.66,621.76 956.76,800.58 957.62,802.23 960.06,806.95 960.5,814.99 958.
↪62,820.64 956.41,827.28 952.06,832.29 945.81,835.41
L 940.61,838
H 480 19.39
Z

M 888.83,786.03
C 888.14,783.99 480.48,78.05 480,78.06 479.53,78.06 71.88,784 71.17,786.05 70.
↪96,786.65 220.55,787 480,787 745.21,787 889.05,786.66 888.83,786.03
Z

M 459.14,763.97
```

```
C 438.41,761.96 415.79,756.9 398,750.29 386.1,745.87 373,739.95 373,738.99␣
 ↪373,738.14 444.62,613.98 445.78,612.82 446.21,612.39 449.71,613.32 453.
 ↪57,614.89 463.16,618.8 469.49,620 480.3,619.99 490.65,619.97 499.03,618.29␣
 ↪507.56,614.52 510.79,613.09 513.73,612.33 514.21,612.81 515.43,614.03␣
 ↪587,738.07 587,738.97 587,739.37 583.96,741.09 580.25,742.79 558.3,752.82␣
 ↪537.23,758.96 512.5,762.55 499.8,764.39 471.32,765.15 459.14,763.97
Z

M 468.63,600.41
C 451.75,596.33 436.88,581.42 433.02,564.72 428.54,545.28 437.25,523.75 453.
 ↪93,513.05 462.22,507.73 469.37,505.72 480,505.72 487.21,505.72 490.93,506.23␣
 ↪495.42,507.86 515.32,515.08 528,532.86 527.96,553.5 527.94,566.9 524.09,576.
 ↪63 514.92,586.43 502.87,599.31 485.61,604.52 468.63,600.41
Z

M 265.53,548.75
C 269.51,513.61 275.97,490.4 289.55,462.5 303.35,434.15 325.23,405.73 348.
 ↪28,386.2 360.54,375.82 372.16,367.76 373.27,368.87 373.78,369.38 390.54,398.
 ↪08 410.5,432.64
L 446.78,495.47 442.13,498.6
C 435.51,503.06 425.73,513.86 421.66,521.21 417.4,528.9 414.45,538.32 413.
 ↪49,547.27
L 412.77,554
H 338.85 264.93
Z

M 546.51,547.27
C 545.55,538.32 542.6,528.9 538.34,521.21 534.39,514.07 524.7,503.26 518.3,498.
 ↪87 515.94,497.24 514,495.5 514,495.01 514,493.93 584.79,371.04 586.46,369.21␣
 ↪588.96,366.49 617.14,389.37 632.82,406.86 668.71,446.89 688.81,493.22 694.
 ↪38,548.75
L 694.91,554
H 621.07 547.23
Z
"""
data_blocks = [b.strip() for b in data.split('\n\n') if b.strip()]

pattern = r'([MLCZHV])\s*([\d\s.,-]+)?'      # ([]) ist Gruppe, speichert in␣
 ↪Gruppe1 den Buchstaben [M oder L oder C] ab     \s* beliebig viele␣
 ↪Leerzeichen  ()Gruppe zwei

plt.close('all')

for i, block in enumerate(data_blocks):
    x=[]
    y=[]
```

```
    for command, values in re.findall(pattern, block):
        # Alle Zahlen im Block extrahieren und in Floats umwandeln
        nums = [float(n) for n in re.findall(r'[-+]?\d*\.?\d+', values.
↪replace(',', ' '))] #[-+]?: negative Zahlen       \d*: beliebig viele␣
↪Ziffern      \.?: optionaler Dezimalpunkt    \d+: Sucht nach mindestens␣
↪einer Ziffer nach dem Punkt und wird von leerzeichen gestoppt

        # elif command == 'C' or command =='c':
        #     # Ein C-Befehl hat 6 Zahlen (x1, y1, x2, y2, x3, y3)
        #     # Wir wollen nur das letzte Paar (Index 4 und 5)
        #     x.append(nums[-2])
        #     y.append(nums[-1])

        if command == 'M' or command == 'L':
            if len(nums) >= 2:
                last_x, last_y = nums[0], nums[1]
                x.append(last_x)
                y.append(last_y)

        elif command == 'C':
                for j in range(0,len(nums),2):
                    last_x, last_y = nums[j], nums[j+1]
                    x.append(last_x)
                    y.append(last_y)
        # # Wir gehen in 6er-Schritten durch die Zahlen (i = 0, 6, 12...)
        #     for j in range(0, len(nums), 6):
        #     # Wir prüfen, ob wir genug Zahlen für ein komplettes Segment haben
        #         if j + 5 < len(nums):
        #         # Der echte Knoten ist immer das letzte Paar einer 6er-Gruppe
        #             x.append(nums[j+4])
        #             y.append(nums[j+5])

        elif command == 'H':
            # Nur X ändert sich, Y bleibt gleich
            if len(nums) >= 1:
                last_x = nums[0]
                x.append(last_x)
                y.append(last_y)

        elif command == 'V':
            # Nur Y ändert sich, X bleibt gleich
            if len(nums) >= 1:
                last_y = nums[0]
                x.append(last_x)
                y.append(last_y)

        elif command == 'Z':
```

```
            if x:
                x.append(x[0])
                y.append(y[0])
                last_x, last_y = x[0], y[0]

    x=np.array(x)
    y=830-np.array(y)

    if i==0 or i>1:
        if i == 0:
            labeltext=r'$E_\alpha$'
        else: labeltext=None
        plt.plot(x, y,color=f"black",linewidth=2, label=labeltext)#color=f"C{1}"
        plt.fill(x, y,color=f"black",)#color=f"C{1}"
    else:
        if i==1:
            labeltext=r'$E_\beta$'        # Nur beim nullten Durchgang das␣
↪Label setzen
        plt.plot(x, y,color='#f9aa00',linewidth=2,␣
↪label=labeltext)#color=f"C{1}"
        plt.fill(x, y, color='#f9aa00',)#color=f"C{1}"



plt.xlabel(r'Z')
plt.ylabel(r'$E_\alpha$ [keV]')
plt.legend()

plt.show()
plt.savefig(Ausgabe_path/"Memefilled.png",format="png")
```


E_α
E_β
800
600
400
200
0
E_α [keV]
0
200
400
600
800
1000
Z